

Synthetic Sensor Trajectory Generation Using Generative Models for Aircraft Predictive Maintenance

Attharv Kumar Jain

Tanil Sheth

Department of Information Technology, K. J. Somaiya School of Engineering,
Somaiya Vidyavihar University, Mumbai, India

Date: 29/08/2026

Github: <https://github.com/tanilxyz/GenAI-for-Aircraft-Sensor-Trajectory>

Abstract—Predictive maintenance for aircraft engines relies on learning degradation patterns from multivariate sensor trajectories and estimating

Remaining Useful Life (RUL). Because complete run-to-failure trajectories are scarce in real maintenance environments, synthetic-data augmentation is an attractive application of Generative AI. This work investigates a Variational Autoencoder (VAE)-based approach for generating synthetic 30-cycle aircraft-engine sensor trajectories and evaluates whether those synthetic samples improve downstream RUL prediction. The NASA C-MAPSS FD001 benchmark is processed with engine-wise train/validation separation, Min-Max normalization, removal of near-constant sensors, and sliding-window segmentation. A sequence VAE with an LSTM encoder and decoder learns a 16-dimensional latent representation. Synthetic trajectories are obtained by perturbing encoded latent vectors with Gaussian noise before decoding. An LSTM RUL regressor is trained in two principal configurations: real trajectories only and an equal-size mixture of real and synthetic trajectories. The actual Colab experiment reports a baseline MAE of 12.078 cycles, RMSE of 16.301 cycles, and R^2 of 0.846. Adding the VAE-generated trajectories increases MAE to 41.075 cycles and RMSE to 46.590 cycles while reducing R^2 to -0.257. Additional experiments with 25%, 50%, and 100% synthetic fractions produce MAE values of 49.498, 43.410, and 39.428 cycles, respectively, all worse than the baseline. Visual analysis shows that generated sensor trajectories are substantially flatter than real trajectories, suggesting that the generator learned average sensor behavior but did not preserve sufficient temporal variability or degradation-specific structure. The study therefore finds that naive VAE augmentation is counterproductive in this configuration and identifies conditional generation, temporal consistency constraints, and low-data evaluation as promising directions.

Index Terms—predictive maintenance, aircraft engines, remaining useful life, NASA C-MAPSS, variational autoencoder, generative AI, synthetic data augmentation, LSTM, time-series generation.

I. INTRODUCTION

Aircraft engines are safety-critical assets whose degradation develops over repeated operating cycles. Maintenance decisions are traditionally supported by scheduled inspections, condition indicators, engineering limits, and historical service knowledge. Data-driven prognostics adds another capability: predicting how much useful operating time remains before a component or engine reaches a failure condition. In this setting, Remaining Useful Life (RUL) prediction converts a sequence of sensor observations into an estimate of the remaining number of operational cycles.

The difficulty is that useful predictive models require representative degradation histories. A fleet may contain many healthy or partially aged engines but relatively few observations of the final stages of failure. Collecting failure data from operational aircraft is also undesirable because the cost of deliberate run-to-failure experimentation is high and safety constraints limit what can be observed. As a result, synthetic-data generation has potential to supplement real trajectories while reducing dependence on rare failure examples.

The NASA Commercial Modular Aero-Propulsion System Simulation (C-MAPSS) benchmark provides a controlled setting for investigating this idea. The FD001 subset contains 100 run-to-failure training trajectories and 100 test trajectories under a single operating condition and single fault mode. Each cycle contains three operating settings and 21 sensor measurements, and the test set includes ground-truth RUL labels.

This paper develops a compact generative pipeline in which a sequence VAE learns latent representations of 30-cycle, 14-sensor windows. Synthetic windows are produced through latent-space perturbation and decoding. The synthetic data are not evaluated in isolation; they are tested by their effect on a separate LSTM RUL predictor. This downstream criterion is important because a visually plausible sample is not necessarily useful for a prognostic task.

The study makes four contributions. First, it implements an end-to-end VAE-based trajectory-generation pipeline suitable for a Google Colab environment. Second, it evaluates synthetic augmentation using the official FD001 test protocol. Third, it quantifies the effect of different synthetic-data fractions. Fourth, it performs qualitative error analysis showing why the generated samples fail to support the RUL predictor in the current configuration.

II. RELATED WORK

A. C-MAPSS and Aircraft Engine Prognostics

C-MAPSS was developed as a high-fidelity engine simulation framework and became a standard benchmark for PHM research. Saxena et al. described the underlying damage-propagation modeling approach in which sensor responses are generated while module efficiency and flow degrade until a failure criterion is reached. The benchmark therefore represents temporal degradation rather than independent classification observations.

NASA documentation specifies that FD001 has one operating condition and one fault mode, while the broader benchmark includes FD002-FD004 with more varied conditions and fault structures. This makes FD001 useful for controlled experiments but also means that performance on FD001 alone should not be interpreted as fleet-wide evidence.

B. Deep Learning for RUL Estimation

LSTM networks are well suited to RUL estimation because their recurrent state can summarize information over a sequence of measurements. Prior aircraft-engine studies have also explored attention mechanisms, convolutional feature extraction, and Transformer-based temporal models. Recent work continues to use C-MAPSS as a benchmark for comparing data-driven RUL approaches.

C. Variational Autoencoders for Synthetic Data

The VAE framework introduced by Kingma and Welling provides a probabilistic latent representation that can be sampled to produce new observations. Unlike a deterministic autoencoder, a VAE encourages the latent space to follow a regularized distribution, supporting controlled generation. For time-series data, recurrent layers can be integrated into the encoder and decoder so that the model learns from sequences rather than isolated points.

D. Synthetic Data for Predictive Maintenance

Synthetic augmentation can be useful when failures are rare, but the generator must reproduce task-relevant properties. In predictive maintenance, these properties include temporal smoothness, sensor correlations, degradation direction, operating-condition effects, and consistency between the trajectory and its RUL label. A generator can minimize reconstruction error while still failing to represent these characteristics. This work therefore evaluates synthetic data through downstream RUL performance instead of using VAE loss alone.

III. RESEARCH PROBLEM AND OBJECTIVES

The research problem is formulated as follows: given a limited set of aircraft-engine degradation trajectories, can a VAE generate additional trajectories that improve RUL prediction when combined with real training data?

The primary hypothesis is H1: adding high-quality synthetic trajectories to the real training set will reduce RUL prediction error. The null or counter-hypothesis is that synthetic trajectories may introduce noise, distribution shift, or label inconsistency and therefore degrade performance.

The specific objectives are to: (1) preprocess the NASA C-MAPSS FD001 data into fixed-length multivariate trajectories; (2)

train a recurrent VAE on real trajectories; (3) synthesize additional trajectories through latent-space perturbation; (4) train an LSTM RUL predictor on real data and on augmented data; (5) compare MAE, RMSE, and R^2 ; and (6) investigate how synthetic-data quantity affects downstream performance.

IV. DATASET AND PREPROCESSING

A. Dataset

The experiment uses NASA C-MAPSS FD001. The loaded files contain 20,631 training rows, 13,096 test rows, and 100 test RUL values. Each row describes one engine unit at one operating cycle. The FD001 setting is intentionally selected because it offers a relatively simple one-condition, one-fault benchmark, allowing the effect of generative augmentation to be studied without confounding operating-condition changes.

Property	Experiment value
Dataset	NASA C-MAPSS FD001
Training observations	20,631
Test observations	13,096
Test engines	100
Initial sensor channels	21
Selected sensors	14
Window length	30 cycles
RUL cap	125 cycles
Train engines	80
Validation engines	20

B. RUL Target

For each training engine, RUL is computed as the maximum observed cycle for that engine minus the current cycle. Targets are then clipped at 125 cycles. The clipping prevents very early-life values from dominating the objective and is consistent with a common FD001 preprocessing choice used in predictive-maintenance benchmarks.

C. Sensor Selection

The 21 raw sensor channels are inspected for variance. Fourteen sensors exceed the variance threshold of 10^{-5} and are retained: s2, s3, s4, s7, s8, s9, s11, s12, s13, s14, s15, s17, s20, and s21. Min-Max normalization is fitted on the training data and applied to validation and test measurements using the same scaling transformation.

D. Windowing and Split Strategy

A sliding window of 30 cycles converts each engine trajectory into overlapping sequences. Each sequence has dimensions 30×14 , and its target is the RUL associated with the final cycle in the window. Before splitting, the training set contains 17,731 sequence windows. An engine-level split assigns the first 80 engine units to training and the remaining 20 to validation, resulting in 13,818 and 3,913 windows. Splitting by engine rather than by individual windows reduces information leakage because neighboring windows from an engine are highly correlated.

V. PROPOSED GENERATIVE-AUGMENTATION METHOD

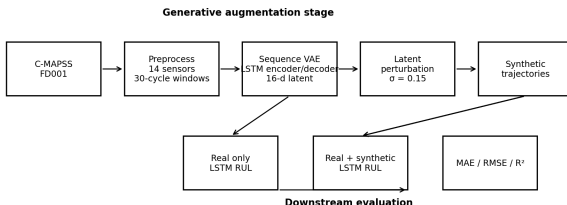


Fig. 1. End-to-end workflow implemented in the notebook: real C-MAPSS windows are encoded into a latent representation, perturbed, decoded into synthetic trajectories, and evaluated through two LSTM RUL predictors.

A. Sequence VAE Architecture

The encoder receives a 30×14 sequence and processes it with a 64-unit LSTM. Two dense layers estimate a 16-dimensional latent mean μ and log-variance $\log \sigma^2$. A latent vector z is obtained using the reparameterization trick. The decoder repeats z across the 30 time steps, applies another 64-unit LSTM, and reconstructs the 14 normalized sensor channels with a time-distributed dense layer using a sigmoid activation.

The VAE is trained with a reconstruction loss plus a scaled KL-divergence regularizer. Conceptually, the objective is $L_{VAE} = L_{reconstruction} + \beta D_{KL}(q(z|x)||p(z))$, where $\beta = 0.001$ in the implementation. The model contains 43,950 trainable parameters and is trained with Adam for up to 30 epochs with batch size 128.

B. Synthetic Trajectory Generation

After training, each real training sequence is encoded to a latent representation. The implementation then adds Gaussian perturbation with standard deviation 0.15 to the sampled latent vector. The perturbed vector is passed through the decoder to obtain a synthetic trajectory. Thus, the generator is not sampling independently from a learned global prior; rather, it creates local variations around latent representations of real trajectories.

This design choice is computationally convenient, but it has an important implication: synthetic samples remain tied to their source trajectory. The implementation assigns the original source sequence's RUL label to the generated sample. The method therefore assumes that the perturbation changes appearance without changing the underlying degradation stage, an assumption that the generated trajectories do not necessarily satisfy.

C. Downstream LSTM Predictor

The RUL predictor contains a 64-unit LSTM returning sequences, dropout of 0.2, a second 32-unit LSTM, a 32-unit ReLU dense layer, and one linear output. The model has 33,729 trainable parameters. Adam with a learning rate of 10^{-3} is used, and early stopping with patience five restores the best validation weights.

D. Experimental Configurations

Configuration	Training data	Purpose
Baseline	13,818 real windows	Reference performance
Augmented	13,818 real + 13,818 synthetic	Equal-size augmentation
25% synthetic	13,818 real + 25% synthetic	Sensitivity analysis
50% synthetic	13,818 real + 50% synthetic	Sensitivity analysis
100% synthetic	13,818 real + 100% synthetic	Sensitivity analysis

E. Evaluation Metrics

MAE is computed as the mean absolute difference between predicted and true RUL. RMSE is the square root of mean squared error and gives greater weight to large failures. R^2 measures the proportion of target variance explained by the predictions. Lower MAE and RMSE and higher R^2 indicate better performance.

VI. EXPERIMENTAL RESULTS

A. VAE Training Behavior

The VAE loss decreases sharply during the first few epochs and then enters a long convergence region. Training and validation curves remain close for most of training, suggesting that the model is not experiencing a severe generalization gap on the reconstruction objective. However, the close training and validation reconstruction losses should not be interpreted as proof that synthetic samples are suitable for RUL prediction.

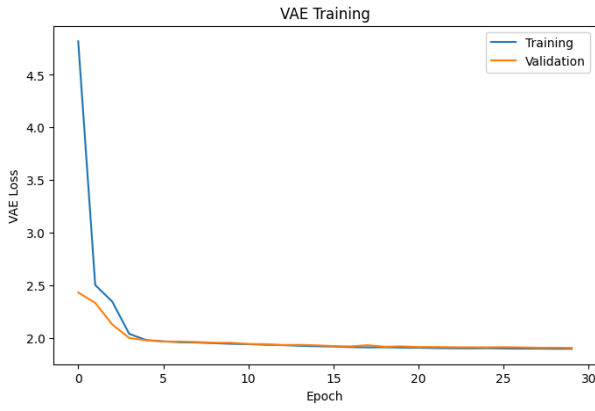


Fig. 2. VAE training and validation loss recorded in the notebook. Loss decreases rapidly early in training and then stabilizes.

B. Synthetic Sample Quality

The real-versus-synthetic plots provide the strongest qualitative evidence about the generator. In the example using sensor s2, the real trajectory fluctuates substantially over the 30-cycle window, while the synthetic trajectory stays close to a nearly constant value. The same pattern appears in three additional examples. The decoder therefore captures a central tendency but not the dynamic amplitude and local changes present in the real observations.

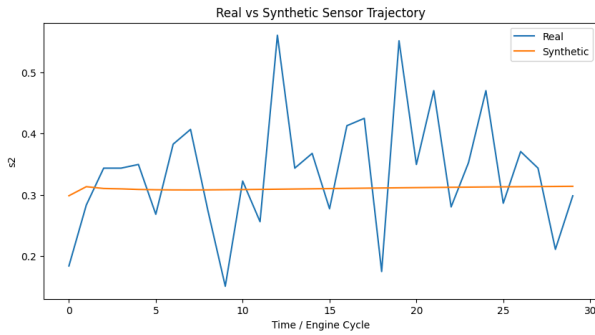


Fig. 3. Single 30-cycle s2 trajectory comparing a real sequence with its synthetic counterpart. The generated sequence is substantially smoothed.

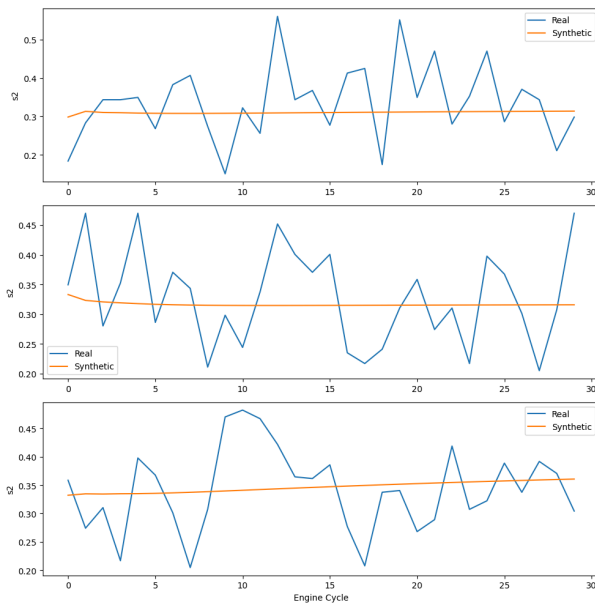


Fig. 4. Three additional real-versus-synthetic s2 trajectories. The repeated flattening of the synthetic curves indicates low temporal diversity.

C. Main RUL Prediction Results

Model	MAE (cycles)	RMSE (cycles)	R ²
LSTM - Real Data	12.078	16.301	0.846
LSTM - Real + VAE Synthetic	41.075	46.590	-0.257

The baseline substantially outperforms the augmented model. MAE increases by 28.996 cycles, equivalent to a 240.06% increase relative to the baseline magnitude. RMSE increases by 30.289 cycles, corresponding to a 185.82% increase. The negative R² indicates that the augmented model performs worse than a simple variance-explaining reference based on the test targets.

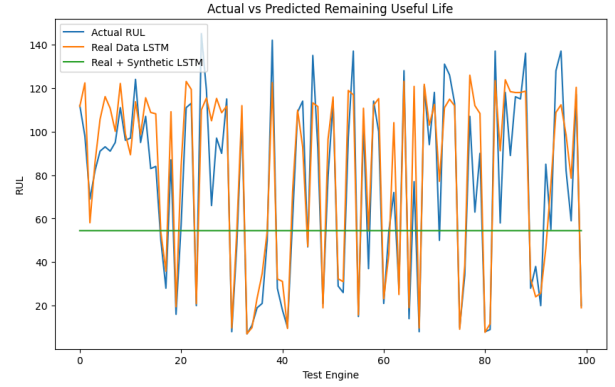


Fig. 5. Actual and predicted RUL for the 100 FD001 test engines. The augmented model is nearly flat and does not follow the engine-to-engine variation of the true RUL.

D. Synthetic-Data Fraction Analysis

The secondary experiment varies the amount of synthetic data while retaining all real training windows. MAE decreases as the synthetic fraction rises: 49.498 cycles at 25%, 43.410 cycles at 50%, and 39.428 cycles at 100%. This within-family trend shows that the augmented model benefits somewhat from additional synthetic examples, but the absolute errors remain far above the 12.078-cycle real-data baseline.

Synthetic fraction	MAE (cycles)	RMSE (cycles)
25%	49.498	58.506
50%	43.410	49.881
100%	39.428	44.184

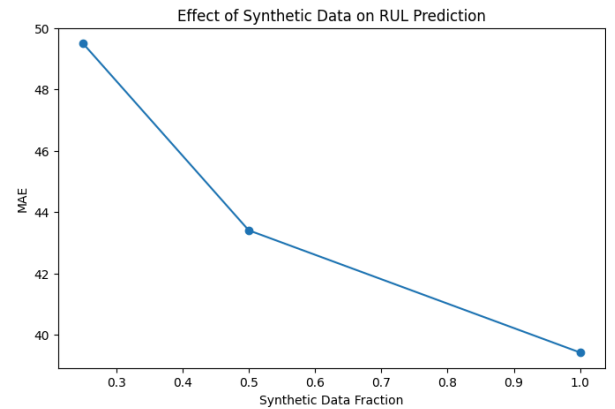


Fig. 6. MAE versus synthetic-data fraction. Increasing synthetic quantity improves the augmented family but does not approach the real-only baseline.

VII. ANALYSIS AND DISCUSSION

A. Why Did the Augmentation Fail?

The central finding is that the VAE learned a representation adequate for reconstruction but not sufficiently informative for the downstream prognostic task. The generated sensor curves collapse toward the mean and discard a large portion of the temporal variation of real trajectories. Because RUL prediction depends on degradation-related trends and interactions across time, removing those variations makes the synthetic samples less informative than real samples.

This is an example of task mismatch between a generative objective and a predictive objective. The VAE minimizes squared reconstruction error over sensor values. A smooth prediction near the conditional mean can reduce average reconstruction loss even when the sample is not a faithful realization of the underlying degradation process. In contrast, an RUL predictor needs trajectories that preserve the relationship between sensor evolution and remaining life.

B. Label Consistency Problem

The implementation copies the RUL label from the source real trajectory to its synthetic counterpart. That label is valid only if the generated sequence represents approximately the same degradation stage. The observed smoothing and amplitude changes make this assumption questionable. A synthetic trajectory may therefore be paired with an RUL target that is inconsistent with its generated sensor values, introducing label noise into the augmented training set.

C. Latent Perturbation Problem

The latent perturbation uses an isotropic Gaussian noise vector with standard deviation 0.15. Nothing in this perturbation explicitly enforces monotonic degradation, sensor correlation, operating-state consistency, or RUL preservation. Consequently, the decoder is free to map a perturbed latent vector to a generic trajectory near the learned average.

D. Interpretation of the Negative Result

A negative experimental result is still useful for research because it narrows the space of viable approaches. The evidence indicates that simply increasing the quantity of low-fidelity synthetic sequences is not sufficient. The 25%-to-100% experiment further shows that even though more synthetic data improves the augmented models relative to one another, the best synthetic configuration remains substantially below the real-data baseline.

The result also supports an important methodological recommendation: generative models for predictive maintenance should be evaluated on both generative fidelity and downstream utility. Visual inspection, reconstruction loss, and latent regularization are not enough. The synthetic data should preserve temporal structure and should produce measurable gains—or at least not harmful degradation—in the actual maintenance prediction task.

VIII. COMPARISON WITH EXISTING C-MAPSS WORK

The baseline result of MAE = 12.078 and RMSE = 16.301 demonstrates that the compact stacked LSTM is capable of learning useful RUL patterns on FD001. Recent benchmark work using more elaborate architectures, including LSTM with attention, Transformers, and TCNs, reports competitive performance under specific protocols. These published results should not be treated as direct head-to-head comparisons with the present experiment because training splits, preprocessing details, test conventions, and hyperparameters may differ.

Model / source	RMSE	MAE	Comment
This work: LSTM, real-only	16.30	12.08	Actual notebook result

This work: LSTM + VAE	46.59	41.07	Actual notebook result
IEEE-IES benchmark: Transformer	12.89	9.71	Different protocol
IEEE-IES benchmark: LSTM + Attention	13.42	10.28	Different protocol

The comparison nonetheless provides context: the baseline is plausible for a compact student implementation, while the synthetic-augmented configuration is clearly not competitive under the experiment's own protocol. The gap is large enough that it cannot reasonably be attributed to minor optimization differences alone.

IX. INDUSTRIAL RELEVANCE

Aircraft predictive maintenance is an industrial problem because maintenance teams must balance reliability, safety, inventory, labor, and aircraft availability. Accurate RUL estimates can support condition-based maintenance by helping engineers identify assets that require closer inspection or earlier intervention. Synthetic trajectory generation could be valuable in situations where historical failures are rare, proprietary, or unevenly distributed.

However, aviation is safety-critical. A synthetic-data system must not be used simply because it creates visually plausible trajectories. The model should be validated against operational measurements and engineering knowledge, and any deployment would require traceability, uncertainty assessment, and domain-specific certification processes. The present C-MAPSS experiment is therefore best interpreted as an algorithmic feasibility study rather than a deployable maintenance system.

The strongest industrial scenario for the proposed method would be a low-data regime in which a maintenance organization has a limited number of failure trajectories and wants to expand a training set for offline model development. In such a setting, a conditional generator that can create trajectories at specific degradation stages may be more useful than the unconditional/local latent perturbation approach tested here.

X. LIMITATIONS

The first limitation is the scope of FD001: one operating condition and one fault mode do not represent the diversity of real aircraft operations. Second, C-MAPSS is simulated data and cannot capture all measurement artifacts, maintenance interventions, sensor recalibration, and environmental effects present in an airline fleet. Third, only 14 of the 21 sensors are used in this implementation, which simplifies the problem but may remove potentially useful degradation signals.

Fourth, the VAE is relatively small and uses a reconstruction-focused objective. Fifth, latent-space perturbation is local and may not generate sufficient diversity. Sixth, the main comparison consists of one training run for each principal configuration, so random initialization and stochastic optimization can affect exact scores. Finally, the evaluation uses MAE, RMSE, and R^2 but does not include the domain-specific asymmetric NASA scoring function.

XI. FUTURE WORK

The most direct improvement is to replace the current VAE with a Conditional VAE (CVAE) conditioned on RUL, normalized age, or degradation stage. This would make generation explicitly dependent on the target condition and could reduce label mismatch. A second improvement is to model all selected sensors jointly with losses that penalize unrealistic sensor correlations and temporal discontinuities.

A third direction is to evaluate purpose-built time-series generators, including conditional GANs and diffusion-based sequence models. These models could be compared under the same downstream RUL predictor and the same augmentation ratios. Generative quality should be evaluated using autocorrelation, cross-sensor correlation, variance preservation, spectral properties, and degradation monotonicity in addition to visual plots.

A fourth direction is to test the hypothesis in a deliberately data-limited regime. For example, models could be trained using only 20%, 40%, and 60% of the available real engines and then compared with synthetic augmentation. This would align the experiment more directly with the industrial motivation for generation: compensating for limited failure data rather than simply increasing an already adequate training set.

Finally, FD002-FD004 and newer NASA turbofan degradation datasets can be used to test robustness across operating conditions and fault modes. Repeated experiments with multiple random seeds and confidence intervals would also make the conclusions more rigorous.

XII. CONCLUSION

This paper presented a VAE-based synthetic sensor trajectory generation pipeline for aircraft predictive maintenance. NASA C-MAPSS FD001 was transformed into 30-cycle multivariate sensor windows using 14 selected sensors. A recurrent VAE learned a 16-dimensional latent representation and generated synthetic trajectories through latent perturbation. A stacked LSTM was then used to predict RUL from the real-only and real-plus-synthetic training sets.

The actual experimental result does not support the initial hypothesis in the current implementation. The real-only LSTM achieved MAE = 12.078 cycles, RMSE = 16.301 cycles, and $R^2 = 0.846$, while the equal-size real-plus-VAE configuration achieved MAE = 41.075 cycles, RMSE = 46.590 cycles, and $R^2 = -0.257$. Synthetic-fraction experiments remained worse than the baseline, although their MAE improved from 49.498 to 39.428 cycles as the synthetic fraction increased from 25% to 100%.

Qualitative trajectory plots explain part of this degradation: synthetic sensor sequences are strongly smoothed toward the mean and do not reproduce the variability of the real time series. Combined with inherited RUL labels and unconditioned latent perturbation, this creates a mismatch between generated sensor patterns and predictive targets. The principal conclusion is therefore that generative augmentation is not automatically beneficial. For aircraft prognostics, future generators must preserve task-relevant temporal and degradation structure, and their success must be judged by downstream RUL performance.

XII. ENGINEERING IMPLICATIONS AND LESSONS LEARNED

The experiment provides several practical lessons for the design of generative predictive-maintenance systems. First, augmentation should target the scarcity that actually limits the supervised task. In this FD001 experiment, the real-only LSTM already achieved an MAE of 12.08 cycles. The data-generation objective therefore had to create additional information rather than simply add more copies of average trajectories. The generated samples did not meet that requirement.

Second, the generator and the downstream predictor should share a notion of the target condition. RUL is not an arbitrary label; it is a physical consequence of degradation. A useful synthetic trajectory should therefore be compatible with an engine's age, degradation stage, and remaining life. Conditioning the generator on these

variables is a natural next step and would turn synthetic generation from generic reconstruction into task-aware data augmentation.

Third, the failure of augmentation demonstrates why visual plausibility is insufficient. The VAE-generated s2 curves look smooth and numerically well behaved, but their low variation is exactly what makes them poor training examples. In a production pipeline, quality checks should include sensor-level variance, temporal autocorrelation, cross-sensor correlation, rate-of-change limits, and consistency with known degradation trajectories before synthetic records are admitted into a training set.

Fourth, the augmentation-ratio results show that the quantity of synthetic data is not the main bottleneck. MAE improved from 49.50 to 39.43 cycles as the synthetic fraction increased from 25% to 100%, but the best augmented result was still far from the 12.08-cycle baseline. This indicates that simply generating more samples from the same weak generator is unlikely to solve the problem. Model quality and conditioning are more important than raw sample count.

Finally, the project illustrates a useful research workflow for Generative AI in engineering: establish a strong non-generative baseline, define a measurable hypothesis, introduce the generative component, and evaluate the generated data by its effect on the real industrial task. This approach prevents a generative model from being judged only by its reconstruction loss and makes the study easier to interpret for maintenance stakeholders.

Observed issue	Evidence in experiment	Recommended remedy
Synthetic smoothing	Generated s2 curves near mean	Temporal/conditional generative loss
RUL-label mismatch	Source RUL copied to synthetic window	Condition generation on RUL/degradation stage
Poor downstream accuracy	MAE 41.07 vs 12.08 cycles	Validate samples using downstream utility
More samples not enough	100% synthetic still MAE 39.43	Improve generator fidelity before scaling
Limited diversity	One synthetic sample per real window	Learn richer latent prior / multiple samples

TABLE V. MAIN FAILURE MODES OBSERVED IN THE IMPLEMENTED GENERATIVE-AUGMENTATION PIPELINE.

APPENDIX A. IMPLEMENTATION AND REPRODUCIBILITY DETAILS

The complete experiment was implemented in Python using TensorFlow/Keras for deep learning, pandas and NumPy for data handling, scikit-learn for normalization and evaluation, and Matplotlib for visualization. The notebook was executed in Google Colab. The procedure is reproducible at the architectural level, but stochastic initialization, VAE sampling, and minibatch optimization can cause variation between runs.

Table IV reports the principal hyperparameters used in the notebook. They are kept fixed across the baseline and augmented RUL predictors so that the main comparison isolates the influence of synthetic trajectories rather than changing the prediction architecture.

The VAE receives a 30×14 input sequence. The encoder LSTM has 64 units, the latent dimension is 16, and the decoder LSTM has 64 units. The reconstruction output uses a sigmoid activation because sensor inputs are normalized to the $[0,1]$ range. The KL-divergence term is weighted by 0.001. The RUL network uses LSTM layers with 64 and 32 units, 0.2 dropout, a 32-unit dense layer, and one linear output neuron.

The synthetic-data process first encodes every real training trajectory, samples a latent representation, and adds Gaussian

perturbation with standard deviation 0.15. The perturbed vector is decoded into a synthetic 30-cycle trajectory. The source trajectory's RUL target is copied to the synthetic sample. This produces 13,818 synthetic windows for the main equal-size augmentation experiment.

Future replications should fix Python, NumPy, and TensorFlow random seeds and repeat each configuration over multiple seeds. The current paper deliberately reports the single-run values stored in the attached notebook rather than replacing them with averaged or estimated values. The principal purpose is to document what the implemented experiment actually produced.

The test set is isolated from generator and predictor training. It is used only after training to evaluate the final RUL estimates. This separation is necessary because the C-MAPSS test trajectories are accompanied by the true RUL values that act as final evaluation targets.

Component	Parameter	Value
Data	Window length	30 cycles
Data	Sensors	14
Data	RUL cap	125 cycles
VAE	Encoder LSTM	64 units
VAE	Latent dimension	16
VAE	Decoder LSTM	64 units
VAE	KL weight	0.001
VAE	Latent noise	$\sigma = 0.15$
VAE	Epochs	30
Training	Batch size	128
RUL model	LSTM units	64 + 32
RUL model	Dropout	0.20
RUL model	Dense layer	32 ReLU
RUL model	Optimizer	Adam, 10^{-3}
Evaluation	Metrics	MAE, RMSE, R^2

TABLE IV. MAIN HYPERPARAMETERS USED IN THE IMPLEMENTED NOTEBOOK.

Algorithm 1 — VAE-based trajectory augmentation and RUL evaluation. 1) Load FD001 train, test, and RUL files. 2) Compute capped training RUL labels. 3) Remove near-constant sensors and fit Min-Max scaling on training data. 4) Create 30-cycle windows. 5) Split 80 engines for training and 20 for validation. 6) Train the sequence VAE. 7) Encode real windows, perturb latent vectors, and decode synthetic windows. 8) Train real-only and real-plus-synthetic LSTM predictors. 9) Evaluate on the unchanged FD001 test set. 10) Repeat augmented training with 25%, 50%, and 100% synthetic fractions.

REFERENCES

[1] A. Saxena, K. Goebel, D. Simon, and N. Eklund, "Damage Propagation Modeling for Aircraft Engine Run-to-Failure Simulation," in Proc. 2008 Int. Conf. on Prognostics and Health Management, Denver, CO, USA, 2008, pp. 1–9, doi: 10.1109/PHM.2008.4711414.

[2] National Aeronautics and Space Administration, "C-MAPSS Jet Engine Simulated Data," NASA Open Data Portal, FD001 dataset documentation, updated May 29, 2025.

[3] D. P. Kingma and M. Welling, "Auto-Encoding Variational Bayes," in Proc. International Conference on Learning Representations (ICLR), 2014.

[4] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," Neural Computation, vol. 9, no. 8, pp. 1735–1780, Nov. 1997, doi: 10.1162/neco.1997.9.8.1735.

[5] L. Liu, X. Song, and Z. Zhou, "Aircraft engine remaining useful life estimation via a double attention-based data-driven architecture," Reliability Engineering & System Safety, vol. 221, 2022, Art. no. 108330, doi: 10.1016/j.ress.2022.108330.

[6] M. Dida, A. Cheriet, and M. Belhadj, "Remaining Useful Life Prediction Using Attention-LSTM Neural Network of Aircraft Engines," International Journal of Prognostics and Health Management, vol. 16, no. 2, 2025, doi: 10.36001/ijphm.2025.v16i2.4274.

[7] F. Isbilen, O. Bektas, and M. Konar, "Deep learning and similarity-based models for predicting turbofan engine remaining useful life: insights from the CMAPSS dataset," The Aeronautical Journal, published online Apr. 14, 2025.

[8] S. M. Elsherif et al., "A deep learning-based prognostic approach for predicting turbofan engine degradation and remaining useful life," Scientific Reports, vol. 15, Art. no. 26251, 2025.

[9] IEEE-IES Industrial-AI Lab, "Industrial Predictive Maintenance: RUL prediction and fault detection on NASA CMAPSS," benchmark implementation repository, accessed 2026.